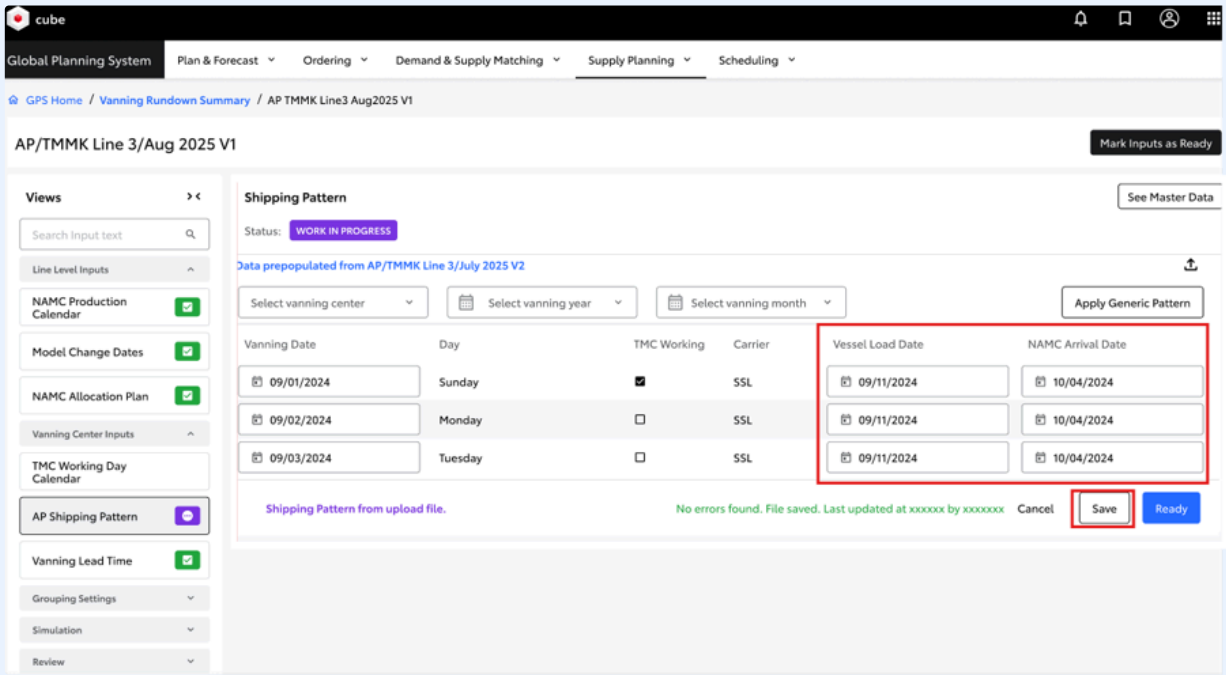
SPPL-195: Shipping Pattern Post API technical workflow

Jira Ticket: SPPL-195: BE | Create technical workflow for Shipping Pattern Get & Post APIs on parameter inputs page including DB details ACCEPTED

This page provides comprehensive information about the POST API for Shipping Pattern on the Input params page, to update shipping details.



Supply Planning POST Shipping Pattern API Details:

1. **Hosted URL**: https://api.spln.cube<env>.toyota.com/<env>
2. **API Name:** <HOSTED_URL>/v/1/1/sp/shipping-pattern
3. **Path Param**: N/A
4. **Query Param**: N/A
5. **Method:** POST
6. **Authorization**: Bearer token (**required**)
7. **Lambda Name**: spln-<env>-update-shipping-pattern-v1-1
8. **API Request Body**:

```
{
  "scenarioId": "uniqueScenarioId",
  "vanningCenter": "NH",
  "userEmail": "user@toyota.com",
  "data": [
    {
      "vanningDate": "2025-01-01",
      "loadDate": "2025-01-11",
```

```
      "arrivalDate": "2025-02-10"
    },
    {
      "vanningDate": "2025-01-02",
      "loadDate": "2025-01-11",
      "arrivalDate": "2025-02-10"
    }
  ]
}
```

9. **API Response:**
   - **Success response with status 200:**

```
{
  "message": "Successfully updated data."
}
```

   - **Error Response:**
     - **any internal server error occurred with status 500:**

```
{
  "errorMessage": "Internal Server Error"
}
```

     - **any unauthorized error occurred with status 401 (Managed by the AWS API Gateway integrated Custom Authorizer):**

```
{
  "message": "Unauthorized"
}
```

     - **If request is in-valid with status 400:**

```
{
  "errorMessage": [<"ValidationError: validation error message”>]
}
```

     - **List of possible validation error message:**
       1. "ValidationError: Request body cannot be empty.”
       2. "ValidationError: scenarioId is required and must be a uuid."
       3. "ValidationError: vanningCenter is required and must be a string."
       4. "ValidationError: userEmail is required and must be a string."
       5. "ValidationError: Scenario doesn't exist."
       6. "ValidationError: Invalid userEmail."
       7. "ValidationError: data is required and must be an array with atleast 1 item."
       8. "ValidationError: vanningDate is required and must be a string in the format YYYY-MM-DD."
       9. "ValidationError: loadDate is required and must be a string in the format YYYY-MM-DD."

10. "ValidationError: arrivalDate is required and must be a string in the format YYYY-MM-DD."
11. "ValidationError: vanningDate must be within the scenario timeframe."
12. "ValidationError: vanningDate must be earlier than the loadDate, and loadDate which must be earlier than the arrivalDate."
13. "ValidationError: Provided vanningDates [<vanningDates>] are non-working days."

10. **Table:**
    a. **scenarios**
    b. **tmc_working_day_calendar**
    c. **shipping_pattern**
    d. **scenario_step_status**

SP Shipping Pattern Post API Workflow:

- **Technical workflow:**
  - Description:
    i. The Supply Planning user logs in to the application.
    ii. After the user provides creates a scenario, opens it and updates Shipping Pattern, a request is sent to the SP Post Shipping Pattern endpoint: ***sp/shipping-pattern*** with request body:

```json
{
  "scenarioId": "uniqueScenarioId",
  "vanningCenter": "NH",
  "userEmail": "user@toyota.com",
  "data": [
    {
      "vanningDate": "2025-01-01",
      "loadDate": "2025-01-11",
      "arrivalDate": "2025-02-10"
    },
    {
      "vanningDate": "2025-01-02",
      "loadDate": "2025-01-11",
      "arrivalDate": "2025-02-10"
    }
  ]
}
```

    iii. After successful authentication, the **spln-update-*shipping-pattern*-v1-1** lambda is triggered by the SP update scenario Shipping Pattern endpoint: ***sp/shipping-pattern***. If authentication fails, an error with status 401 (*refer: error response-**any unauthorized error occurred with status 401***) will be returned and sent to client.

iv. After user authentication has been confirmed, the **spln-update-*shipping-pattern*-v1-1** lambda will perform validation on the request payload. If payload is valid will proceed to next step else throw validation error (*refer: error response**: Section if request is in-valid with status 400 and List of possible validation error message***). Below queries are used to check if provided scenario is valid:

```
select * from supply_planning.scenarios where scenario_id = :scenarioId and
is_active = true;
```

v. After the request payload is successfully validated, the **sp-update-*shipping-pattern*-v1-1** lambda execute logic to generate shipping pattern data based on input and below query to post scenario Shipping Pattern data into the ***shipping-pattern*** table.

```
await prisma.$executeRaw`
UPDATE supply_planning.shipping_pattern sp
SET
  vessel_load_date  = src.vessel_load_date,
  namc_arrival_date = src.namc_arrival_date,
  updated_by        = ${userEmail}::text,
  last_updated_timestamp = NOW()
FROM (
  VALUES
    ${Prisma.join(
      input.map(item => Prisma.sql`(
        ${item.vanningCenter}::text,
        ${item.vanningDate}::date,
        ${item.vesselLoadDate}::date,
        ${item.namcArrivalDate}::date
      )`)
    )}
) AS src(
  vanning_center,
  vanning_date,
  vessel_load_date,
  namc_arrival_date
)
WHERE sp.scenario_id    = ${scenarioId}::uuid
  AND sp.vanning_center = src.vanning_center
  AND sp.vanning_date   = src.vanning_date
`;
```

vi. Update the step & scenario status to “In Progress”. Only update scenario status to “In Progress” if its not already using scenarioData from query (iv) response.

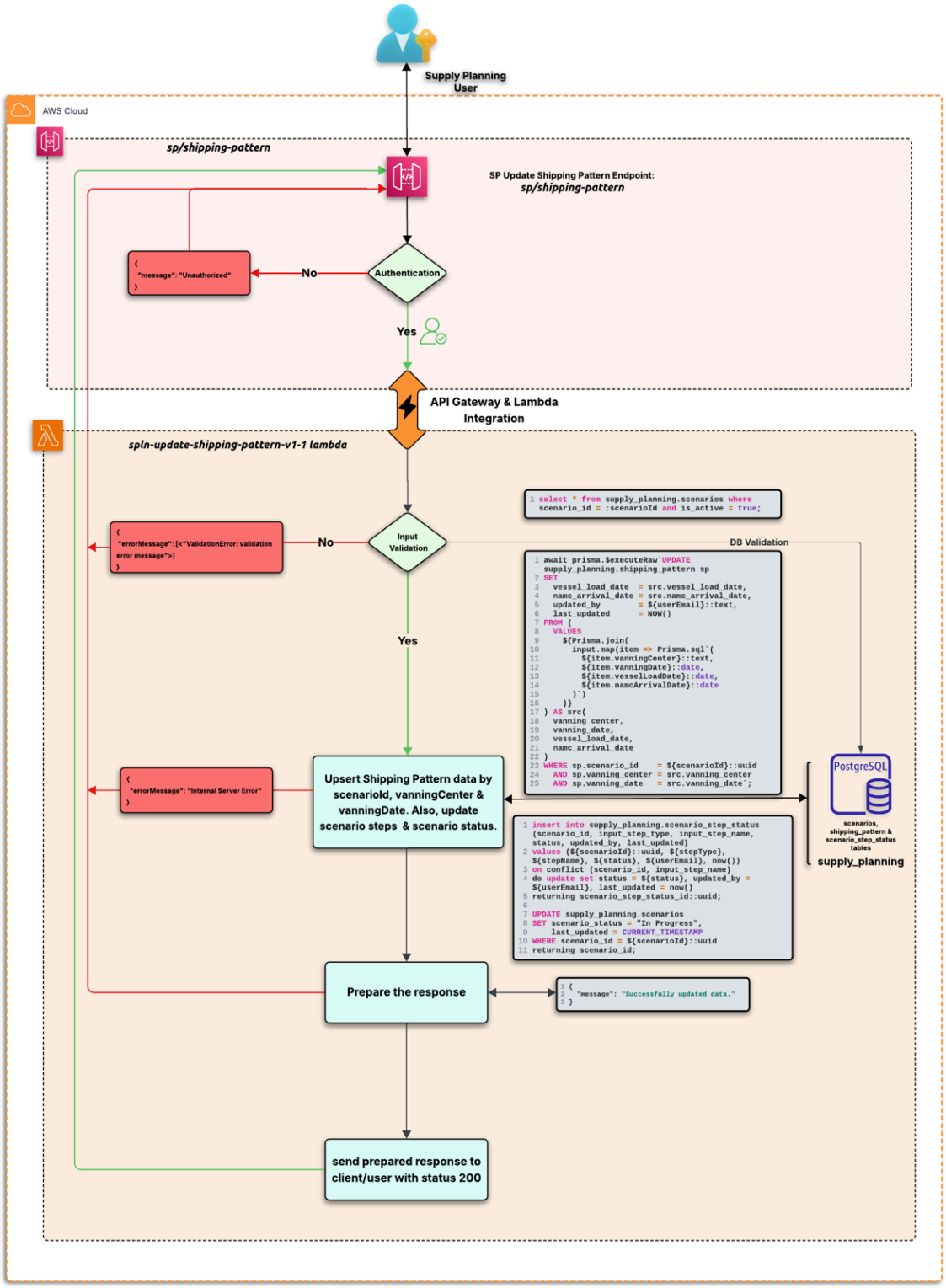
```
insert into supply_planning.scenario_step_status (scenario_id, input_step_type,
input_step_name, status, created_by, created_date_timestamp)
values (${scenarioId}::uuid, ${stepType}, ${stepName}, ${status}, ${userEmail},
now())
on conflict (scenario_id, input_step_name)
do update set status = ${status}, updated_by = ${userEmail},
last_updated_timestamp = now()
returning scenario_step_status_id::uuid;

UPDATE supply_planning.scenarios
SET scenario_status = ${scenarioStatus},
    updated_by = ${userEmail},
    last_updated_timestamp = CURRENT_TIMESTAMP
```

```
WHERE scenario_id = ${scenarioId}::uuid
returning scenario_id;
```

vii. After updating the Shipping Pattern data, return the below response to the client with a status code of 200:

```
{
  "message": "Successfully updated data."
}
```

viii. If there is an error while updating the data, an “Internal Server Error” with a status code of 500(*refer: error response-**any internal server error occurred with status 500***) will be returned. Otherwise, a successful response with a status code of 200 will be sent.



SP Shipping Pattern Post API Techncial workflow

Queries: